2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：	C++ 简单程序设计（实验一）		
实验日期：	2026.10.23	实验地点：	于刚宋晓楼 A237
学　　号：	2025302183225	姓　　名：	徐维鑫
学　　院：	人工智能学院	专　　业：	人工智能专业
指导老师：	丁建立		

一、实验目的

1. 学会编写简单的 C++ 程序；
2. 掌握基本数据类型变量和常量的应用；
3. 掌握运算符和表达式的应用；
4. 掌握结构化程序设计基本控制结构的运用；
5. 掌握简单的输入输出语句的运用；
6. 掌握头文件的运用；
7. 熟悉调试功能：单步执行、设置断点、观察变量值。

二、实验环境

操作系统：Windows 11（版本 10.0.26200）。

编译器:MinGW-w64 GCC 8.1.0(g++),编译选项 -std=c++11 -Wall -Wextra。

编辑器与终端：Visual Studio Code、Windows 命令提示符。

项目组织：每个题目单独建立一个 .cpp 源文件，单独编译为可执行文件后运行。

三、实验过程及结果

本次实验按照课件的四个课堂练习展开，共完成 22 个程序。每个题目单独建立一个源文件并单独编译运行，全部源码位于仓库 lab01/src/ 目录，下面按练习顺序给出各题目的代码、运行结果与分析。

练习一　基本数据类型

1. 读入并显示整数（补充 2-1）

实验内容：从键盘两次读入半径，观察变量值在程序运行过程中的变化。

代码：

```
// 练习 1-1 读入并显示整数（课件 补充 2-1）
// 观察：cin 从键盘读入数据；变量 radius 的值在运行过程中可以改变。
#include <iostream>
using namespace std;

int main()
{
    int radius;
    cout << "Please enter the radius!\n";
    cin >> radius;
    cout << "The radius is:" << radius << '\n';
    cout << "PI is:" << 3.14 << '\n';
    cout << "Please enter a different radius!\n";
    cin >> radius;
    cout << "Now the radius is changed to:" << radius << '\n';
    return 0;
}
```

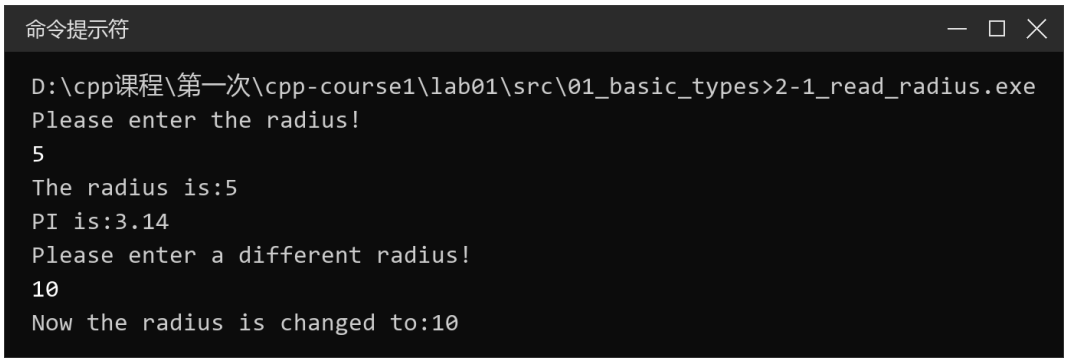


图 1-1　读入并显示整数（补充 2-1）运行结果

结果分析：第一次输入 5，输出 radius 为 5；第二次输入 10，输出变为 10。cin >> radius 会用新读入的值覆盖变量原来的值，说明普通变量的值在运行期间可以改变。

2. 为常量命名（补充 2-2）

实验内容：用 const 定义符号常量 pi，观察其值是否会被后续操作改变。

代码：

```
// 练习 1-2 为常量命名（课件 补充 2-2）
// 观察：符号常量 pi 在运行期间不能修改；去掉 cin >> pi 的注释后编译报错。
#include <iostream>
using namespace std;

int main()
{
    const double pi(3.14159);
    int radius;
    cout << "Please enter the radius!\n";
    cin >> radius;
    cout << "The radius is:" << radius << '\n';
    cout << "PI is:" << pi << '\n';
    cout << "Please enter a different radius!\n";
    cin >> radius;
    cout << "Now the radius is changed to:" << radius << '\n';
    cout << "PI is still:" << pi << '\n';
    // cin >> pi;   // 常量不能被赋值，取消注释将产生编译错误
    return 0;
}
```

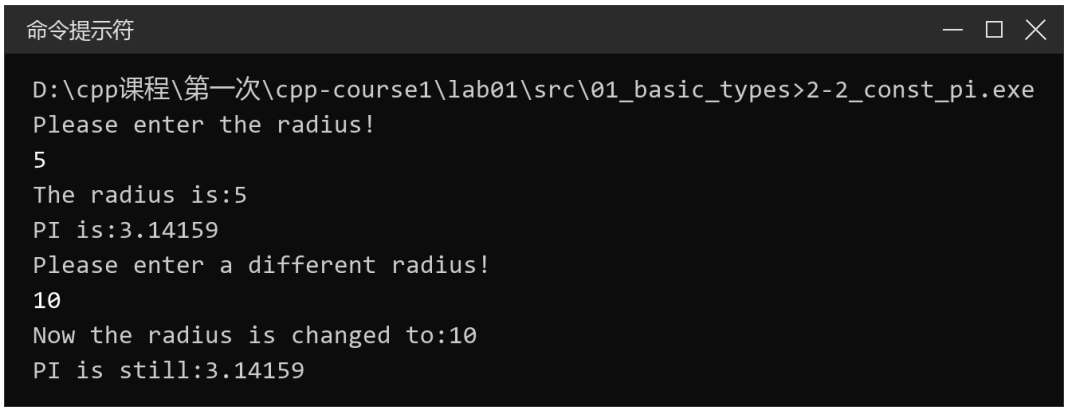


图 1-2 为常量命名（补充 2-2）运行结果

结果分析：两次输出 pi 均为 3.14159。const 常量在运行期间不能被修改，若取消 cin >> pi 的注释，编译会报错；定义 const 变量时必须初始化。

3. 变量的初始化（补充 2-3）

实验内容：在定义变量时把 radius 初始化为 0，再读入新值。

代码：

```
// 练习 1-3 变量的初始化（课件 补充 2-3）
// 观察：在定义变量时直接初始化，可以避免使用未初始化的随机值。
#include <iostream>
using namespace std;

int main()
{
    const double pi(3.14159);
    int radius(0);
    cout << "The initial radius is:" << radius << '\n';
    cout << "PI is:" << pi << '\n';
    cout << "Please enter a different radius!\n";
    cin >> radius;
    cout << "Now the radius is changed to:" << radius << '\n';
    cout << "PI is still:" << pi << '\n';
    return 0;
}
```



图 1-3 变量的初始化（补充 2-3）运行结果

结果分析：初始 radius 为 0，输入 10 后变为 10。未初始化的局部变量其值是不确定的，定义变量时应养成初始化的习惯。

4. 有符号整数与无符号整数（补充 2-4）

实验内容：把 50000 赋给 unsigned short 变量，再赋给 short 变量，观

察溢出。

代码：

```
// 练习 1-4 整数变量的定义与输出（课件 补充 2-4）
// 观察：50000 超出 short 的表示范围（-32768~32767），赋值后发生溢出。
#include <iostream>
using namespace std;

int main()
{
    short int i;
    unsigned short int j;
    j = 50000;
    i = j;
    cout << "Short int is:" << i << endl;
    cout << "Short unsigned int is: " << j << endl;
    return 0;
}
```



图 1-4 有符号整数与无符号整数（补充 2-4）运行结果

结果分析：unsigned short 正确输出 50000；short 的上限是 32767，50000 超出范围，按 16 位补码解释为 50000 − 65536 = −15536，这就是数值溢出。

5. 不同类型整数的最值（补充 2-5）

实验内容：输出 <climits> 中定义的各整数类型最小值与最大值。

代码：

```
// 练习 1-5 不同类型整数的最值（课件 补充 2-5）
// 观察：<climits> 中定义的符号常量给出了各整数类型的最小值和最大值。
#include <iostream>
#include <climits>
using namespace std;

int main()
{
    cout << "Min of short is: " << SHRT_MIN << endl;
    cout << "Max of short is: " << SHRT_MAX << endl;
    cout << "Min of int is: " << INT_MIN << endl;
    cout << "Max of int is: " << INT_MAX << endl;
    cout << "Min of long is: " << LONG_MIN << endl;
    cout << "Max of long is: " << LONG_MAX << endl;
    cout << "Max of unsigned short is: " << USHRT_MAX << endl;
    cout << "Max of unsigned int is: " << UINT_MAX << endl;
    cout << "Max of unsigned long is: " << ULONG_MAX << endl;
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\01_basic_types>2-5_integer_limits.exe
Min of short is: -32768
Max of short is: 32767
Min of int is: -2147483648
Max of int is: 2147483647
Min of long is: -2147483648
Max of long is: 2147483647
Max of unsigned short is: 65535
Max of unsigned int is: 4294967295
Max of unsigned long is: 4294967295
```

图 1-5 不同类型整数的最值（补充 2-5）运行结果

结果分析：本机 short 占 2 字节（−32768～32767），int 和 long 均占 4 字节，unsigned 类型最小值为 0。long 与 int 同为 4 字节是 Windows 平台的特点。

6. 整型文字常量的应用（补充 2-6）

实验内容：分别以十进制、八进制、十六进制输入 123，再以三种进制输出。

代码：

```
// 练习 1-6 整型文字常量的应用（课件 补充 2-6）
// 观察：dec / oct / hex 操纵符控制输入输出的进制，
//       123、0173、0x7B 是同一个数的不同进制写法。
#include <iostream>
using namespace std;

int main()
{
    int isample, osample, hsample;
    unsigned long ulsample;

    cin >> isample >> oct >> osample >> hex >> hsample;
    cout << isample << ';' << oct << osample << ';' << hex << hsample << endl;

    isample = 123;
    osample = 0173;
    hsample = 0x7B;
    ulsample = 4294967295UL;

    cout << dec << isample << ';' << oct << isample
         << ';' << hex << isample << endl;
    cout << dec << osample << ';' << oct << osample
         << ';' << hex << osample << endl;
    cout << dec << hsample << ';' << oct << hsample
         << ';' << hex << hsample << endl;
    cout << dec << ulsample << ';' << oct << ulsample
         << ';' << hex << ulsample << endl;
    return 0;
}
```



图 1-6 整型文字常量的应用（补充 2-6）运行结果

结果分析：输入的 123、173（八进制）、7B（十六进制）是同一个数，输出均为 123;173;7b。dec、oct、hex 只改变输入输出的进制表示，不改变数值本身。

7. 不同类型浮点数的应用（补充 2-7）

实验内容：分别用 float、double、long double 计算半径 5.5 的圆面积。

代码：

```
// 练习 1-7 不同类型浮点数的应用（课件 补充 2-7）
// 观察：float / double / long double 的有效位数和 sizeof 大小不同。
#include <iostream>
using namespace std;

const float PI_FLOAT = 3.1415926f;
const double PI_DOUBLE = 3.1415926;
const long double PI_LDOUBLE = 3.1415926L;   // 建议加 L 后缀

int main()
{
    float nRadiusFloat = 5.5f, nAreaFloat;
    double nRadiusDouble = 5.5, nAreaDouble;
    long double nRadiusLDouble = 5.5L, nAreaLDouble;

    nAreaFloat = PI_FLOAT * nRadiusFloat * nRadiusFloat;
    nAreaDouble = PI_DOUBLE * nRadiusDouble * nRadiusDouble;
    nAreaLDouble = PI_LDOUBLE * nRadiusLDouble * nRadiusLDouble;

    cout << "nAreaFloat = " << nAreaFloat
         << " , sizeof(nAreaFloat) = " << sizeof(nAreaFloat) << endl;
    cout << "nAreaDouble = " << nAreaDouble
         << " , sizeof(nAreaDouble) = " << sizeof(nAreaDouble) << endl;
    cout << "nAreaLDouble = " << nAreaLDouble
         << " , sizeof(nAreaLDouble) = " << sizeof(nAreaLDouble) << endl;
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\01_basic_types>2-7_float_types.exe
nAreaFloat = 95.0332 , sizeof(nAreaFloat) = 4
nAreaDouble = 95.0332 , sizeof(nAreaDouble) = 8
nAreaLDouble = 95.0332 , sizeof(nAreaLDouble) = 16
```

图 1-7 不同类型浮点数的应用（补充 2-7）运行结果

结果分析：三种类型算出的面积相同（在有效位数范围内），但 sizeof 分别为 4、8、16 字节。float 的有效数字约 7 位，double 约 15 位，精度要求高时应使用 double。

8. 字符数据的应用（补充 2-8）

实验内容：输出普通字符，并观察制表符、退格符、响铃符的效果。

代码：

```
// 练习 1-8 字符数据的应用（课件 补充 2-8）
// 观察：字符常量、字符串常量以及 \t \b \a 等转义序列的效果。
#include <iostream>
using namespace std;

int main()
{
    cout << 'A' << ' ' << 'a' << endl;   // 输出普通字符
    cout << "one\ttwo\tthree\n";         // 使用水平制表符
    cout << "123\b\b45\n";               // 使用退格符
    cout << "Alert\a\n";                 // 使用响铃符
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\01_basic_types>2-8_char_data.exe
A a
one     two     three
145
Alert
```

图 1-8 字符数据的应用（补充 2-8）运行结果

结果分析：\t 使输出按制表位对齐；123\b\b45 中两个退格把光标移回 1 之后，45 覆盖了 23，实际显示为 145；\a 触发响铃，一般终端中没有声音。

9. bool 类型的应用（补充 2-9）

实验内容：定义 bool 变量，观察 bool 与 int 之间的转换。

代码：

```
// 练习 1-9 定义 bool 类型变量并进行算术运算（课件 补充 2-9）
// 观察：bool 输出为 1/0；整数 0 转换为 false，非 0 值转换为 true。
#include <iostream>
using namespace std;

int main()
{
    bool bV1 = true, bV2 = false;
    cout << "bool value bV1 = " << bV1 << endl;
    cout << "bool value bV2 = " << bV2 << endl;

    int nV1 = bV1, nV2 = 0;
    bV1 = nV2;
    cout << "int value nV1 = " << nV1 << endl;
    cout << "bool value bV1 = " << bV1 << endl;
    return 0;
}
```



图 1-9 bool 类型的应用（补充 2-9）运行结果

结果分析：bool 值输出为 1/0；true 参与算术运算时转换为 1，整数 0 赋给 bool 得 false，所有非 0 值转换为 true。

练习二　算术运算符和赋值运算符

1. 算术运算（补充 2-10）

实验内容：测试整数除法、取模、混合类型运算以及前置/后置自增自减。

代码：

```
// 练习 2-1 算术运算（课件 补充 2-10）
// 观察：整数相除取整、% 只能用于整数、++/-- 前置与后置的区别。
#include <iostream>
using namespace std;

int main()
{
    int val1 = 24;
    int val2 = 5;
    double val3 = 24;
    double val4 = 5;

    cout << "int/int, 24/5= " << val1 / val2 << endl;
    cout << "int/int, 24%5= " << val1 % val2 << endl;

    val2 = -5;
    cout << "int/int, 24%(-5)= " << val1 % val2 << endl;

    cout << "double/double, 24/5= " << val3 / val4 << endl;
    // cout << "double/double, 24%5= " << val3 % val4 << endl;  // 浮点数不能取模

    cout << "double/int, 24/5= " << val3 / val2 << endl;
    cout << "int/double, 24/5= " << val1 / val4 << endl;

    val1 = 5;
    cout << "val1 = " << val1 << endl;
    cout << val1++ << ", ";
    cout << ++val1 << ", ";
    cout << val1-- << ", ";
    cout << --val1 << endl;

    val1 = 5;
    cout << "val1 = " << val1 << endl;
    cout << val1++ << ", " << ++val1 << ", "
         << val1-- << ", " << --val1 << endl;
    // 注意：同一表达式中多次修改同一变量，求值顺序因编译器而异
    return 0;
}
```

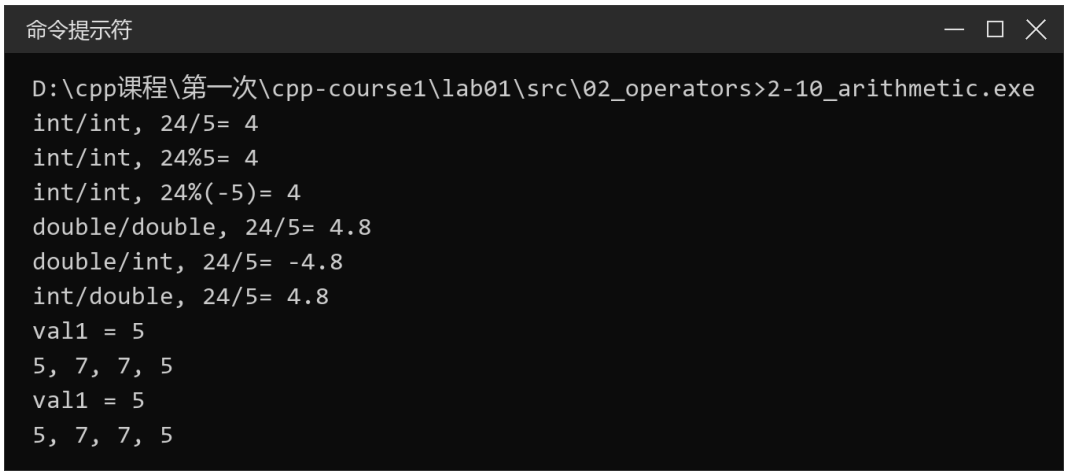


图 2-1 算术运算（补充 2-10）运行结果

结果分析：24/5 得 4（小数被截掉），24%5 得 4，24%(-5) 仍得 4（余数符号随被除数）；double 运算得 4.8。val1++、++val1、val1--、--val1 依次输出 5、7、7、5。最后一行在同一表达式中多次修改 val1，g++ 给出 -Wsequence-point 警告，属于未定义行为，不同编译器结果可能不同。

2. 赋值运算（补充 2-11）

实验内容：观察连续赋值、赋值表达式的值，以及 ++ 与负号、加法结合时

的运算顺序。

代码：

```
// 练习 2-2 赋值运算的应用（课件 补充 2-11）
// 观察：赋值表达式的值就是赋给左边变量的值，因此可以连续赋值。
#include <iostream>
using namespace std;

int main()
{
    int ival1, ival2;
    double fval;
    char cval;

    ival1 = 1;
    ival2 = 2;
    cout << "ival1= " << ival1 << endl;
    cout << "ival2= " << ival2 << endl;

    ival1 = ival2 = 0;
    ival1 = fval = 0;
    ival1 = cval = 'a';
    cout << "ival1= " << ival1 << endl;
    cout << "ival2= " << ival2 << endl;
    cout << "fval= " << fval << endl;
    cout << "cval= " << cval << endl;

    int ival3 = fval = 8;
    cout << "ival3= " << ival3 << endl;
    cout << "fval= " << fval << endl;
    cout << "ival1 = " << ival1 << ", ival2 = " << ival2 << endl;

    ival2 = -ival1++;
    cout << "ival2 = -ival1++, ival1= " << ival1
         << ", ival2 = " << ival2 << endl;

    ival1 = ++ival2 + ival1;
    cout << "ival1 = ++ival2+ival1, ival1= " << ival1 << endl;
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\02_operators>2-11_assignment.exe
ival1= 1
ival2= 2
ival1= 97
ival2= 0
fval= 0
cval= a
ival3= 8
fval= 8
ival1 = 97, ival2 = 0
ival2 = -ival1++, ival1= 98, ival2 = -97
ival1 = ++ival2+ival1, ival1= 2
```

图 2-2 赋值运算（补充 2-11）运行结果

结果分析：赋值运算符从右向左结合，ival1 = cval = 'a' 使 ival1 得到

字符 a 的编码 97；ival2 = -ival1++ 先把 ival1 的相反数赋给 ival2，再让 ival1 自增；++ival2 + ival1 中前置 ++ 优先级最高，先自增再相加，最后赋值。

练习三　关系/逻辑运算符、sizeof、位运算

1. 比较数据并输出结果（补充 2-12）

实验内容：用关系运算、相等运算和逻辑运算比较四个整数。

代码：

```
// 练习 3-1 比较数据并输出结果（课件 补充 2-12）
// 观察：关系运算、相等运算、逻辑运算的结果都是 bool 类型，输出为 1/0。
#include <iostream>
using namespace std;

int main()
{
    int ival1 = 1, ival2 = 2, ival3 = 3, ival4 = 4;
    bool nFlag;

    nFlag = ival1 == ival2 ? true : false;
    cout << "Is 1 equals 2?: " << nFlag << endl;

    nFlag = ival1 < ival2 ? true : false;
    cout << "Is 1 less than 2?: " << nFlag << endl;

    nFlag = (ival1 < ival2 && ival3 < ival4) ? true : false;
    cout << "Is 1 less than 2 and 3 less than 4: " << nFlag << endl;
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\03_logic_sizeof_bitwise>2-12_compare.exe
Is 1 equals 2?: 0
Is 1 less than 2?: 1
Is 1 less than 2 and 3 less than 4: 1
```

图 3-1　比较数据并输出结果（补充 2-12）运行结果

结果分析：1 == 2 为假输出 0，1 < 2 为真输出 1，两个关系式相与仍为真输出 1。关系、相等、逻辑运算的结果都是 bool 类型，默认输出为 1/0。

2. sizeof 运算符的应用（补充 2-13）

实验内容：输出各基本数据类型在当前编译环境下的字节数。

代码：

```
// 练习 3-2 sizeof 运算符的应用（课件 补充 2-13）
// 观察：各基本数据类型在当前编译环境下的字节数。
#include <iostream>
#include <string>
using namespace std;

int main()
{
    cout << "sizeof(short)= " << sizeof(short) << endl;
    cout << "sizeof(unsigned short)= " << sizeof(unsigned short) << endl;
    cout << "sizeof(int)= " << sizeof(int) << endl;
    cout << "sizeof(unsigned int)= " << sizeof(unsigned int) << endl;
    cout << "sizeof(long)= " << sizeof(long) << endl;
    cout << "sizeof(unsigned long)= " << sizeof(unsigned long) << endl;
    cout << "sizeof(float)= " << sizeof(float) << endl;
    cout << "sizeof(double)= " << sizeof(double) << endl;
    cout << "sizeof(long double)= " << sizeof(long double) << endl;
    cout << "sizeof(char)= " << sizeof(char) << endl;
    return 0;
}
```

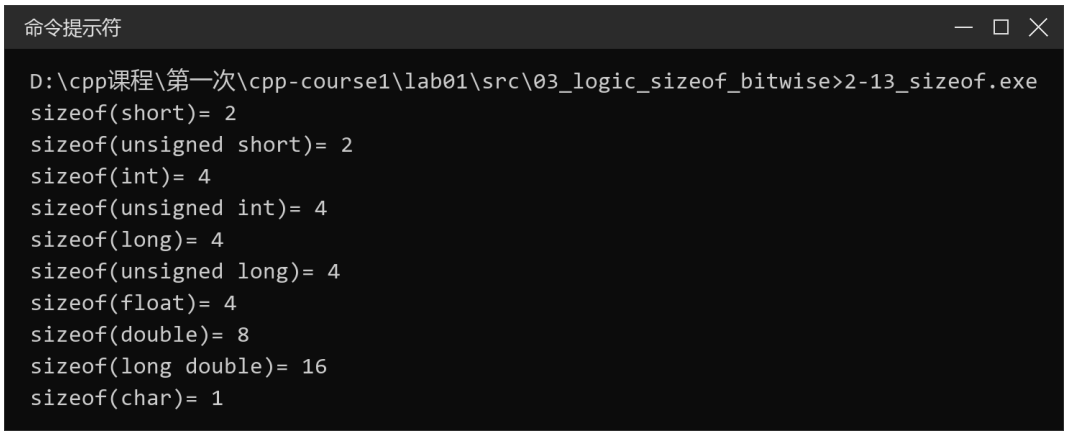


图 3-2 sizeof 运算符的应用（补充 2-13）运行结果

结果分析：short 2 字节、unsigned short 2 字节、int 4 字节、unsigned int 4 字节、long 4 字节、unsigned long 4 字节、float 4 字节、double 8 字节、long double 16 字节、char 1 字节。sizeof 在编译期求值，是获取类型长度的可移植方法。

3. 位运算符的应用（补充 2-14）

实验内容：演示按位取反、与、异或、或，以及置位、清位和逐位翻转。

代码：

```
// 练习 3-3 位运算符的应用（课件 补充 2-14）
// 观察：按位与置 0、按位或置 1、按位异或翻转指定二进制位。
#include <iostream>
#include <bitset>
using namespace std;

int main()
{
    cout << "~15 = " << (~15) << endl;
    cout << "15 & 21 = " << (15 & 21) << endl;
    cout << "15 ^ 21 = " << (15 ^ 21) << endl;
    cout << "15 | 21 = " << (15 | 21) << endl;

    unsigned int nTest = 9;
    cout << "nTest = " << nTest << endl;

    nTest |= 1 << 4;                    // 将第 4 位置为 1
    cout << "After set the position 4 to 1, nTest = " << nTest << endl;

    nTest &= ~(1 << 4);                 // 将第 4 位置为 0
    cout << "After set the position 4 to 0, nTest = " << nTest << endl;

    bool nFlag;
    for (int i = 0; i < 16; i++) {      // 将低 16 位逐位翻转
        nFlag = nTest & (1 << i);
        if (nFlag)
            nTest &= ~(1 << i);
        else
            nTest |= 1 << i;
    }
    cout << "After flip, nTest = " << nTest << endl;
    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\03_logic_sizeof_bitwise>2-14_bitwise.exe
~15 = -16
15 & 21 = 5
15 ^ 21 = 26
15 | 21 = 31
nTest = 9
After set the position 4 to 1, nTest = 25
After set the position 4 to 0, nTest = 9
After flip, nTest = 65526
```

图 3-3 位运算符的应用（补充 2-14）运行结果

结果分析：~15 = −16（补码表示），15&21 = 5，15^21 = 26，15|21 = 31。nTest = 9（1001B）第 4 位置 1 后为 25（11001B），清位后回到 9；低 16 位全部翻转后为 65526（0xFFF6）。

练习四　结构化程序设计的控制结构

1. 企业奖金提成

实验内容：输入当月利润（万元），按六个区间分段累进计算应发奖金。

代码：

```
// 练习 4-1 企业奖金提成
// 利润 I 分 6 档，按分段累进方式计算奖金（输入、输出单位均为万元）。
#include <iostream>
using namespace std;

int main()
{
    double profit;  // 当月利润，单位：万元
    cout << "请输入当月利润（万元）：";
    cin >> profit;

    if (profit < 0) {
        cout << "利润不能为负数。" << endl;
        return 1;
    }

    double bonus;
    if (profit <= 10)
        bonus = profit * 0.10;
    else if (profit <= 20)
        bonus = 10 * 0.10 + (profit - 10) * 0.075;
    else if (profit <= 40)
        bonus = 10 * 0.10 + 10 * 0.075 + (profit - 20) * 0.05;
    else if (profit <= 60)
        bonus = 10 * 0.10 + 10 * 0.075 + 20 * 0.05 + (profit - 40) * 0.03;
    else if (profit <= 100)
        bonus = 10 * 0.10 + 10 * 0.075 + 20 * 0.05 + 20 * 0.03
                + (profit - 60) * 0.015;
    else
        bonus = 10 * 0.10 + 10 * 0.075 + 20 * 0.05 + 20 * 0.03
                + 40 * 0.015 + (profit - 100) * 0.01;

    cout << "应发放奖金：" << bonus << " 万元（" << bonus * 10000 << " 元）" << endl;
    return 0;
}
```

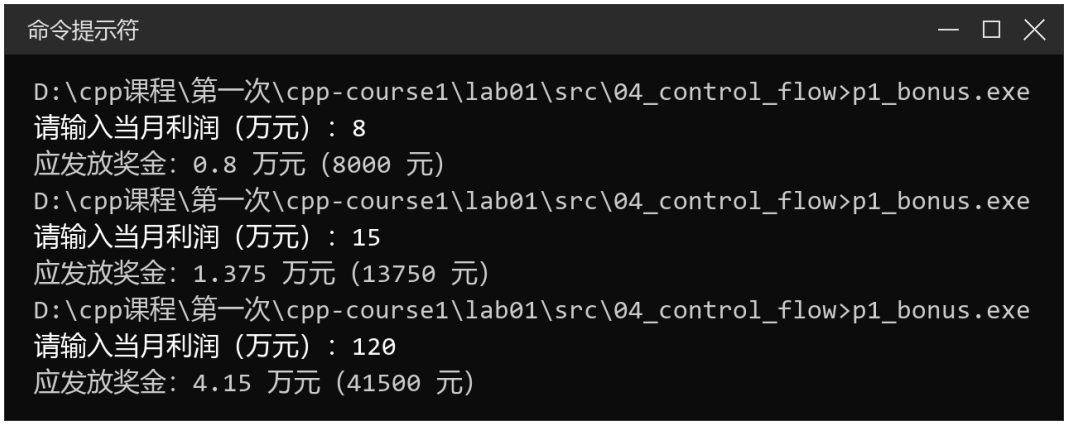


图 4-1　企业奖金提成运行结果

结果分析：利润 8 万元时奖金为 0.8 万元；15 万元时为 1 + 0.375 = 1.375 万元；120 万元时为 3.95 + 0.2 = 4.15 万元。分段累进的关键是：前面各档按满额计算，只有超出当前档起点的部分才按当前档比例提成。

2. 根据电影评分输出评价

实验内容：输入 0～10 的整数评分，按区间输出“经典”“非常好”“一般”“烂片”。

代码：

```
// 练习 4-2 根据电影评分输出评价
// 10~9 经典，8~7 非常好，6~5 一般，5 以下烂片。
#include <iostream>
using namespace std;

int main()
{
    int score;
    cout << "请输入电影评分（0~10）：";
    cin >> score;

    if (score < 0 || score > 10) {
        cout << "评分无效，请输入 0~10 之间的整数。" << endl;
        return 1;
    }

    if (score >= 9)
        cout << "你认为电影经典" << endl;
    else if (score >= 7)
        cout << "你认为电影非常好" << endl;
    else if (score >= 5)
        cout << "你认为电影一般" << endl;
    else
        cout << "你认为电影是烂片" << endl;

    return 0;
}
```

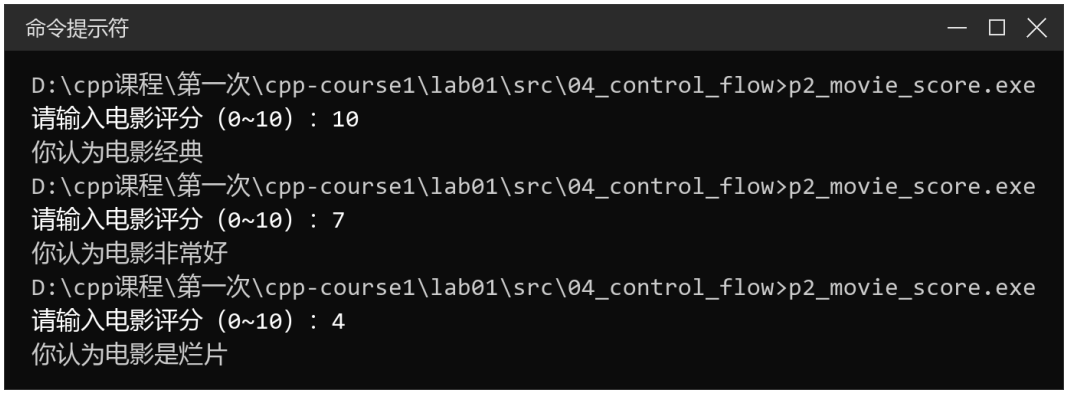


图 4-2 根据电影评分输出评价运行结果

结果分析：10 分输出“经典”，7 分输出“非常好”，4 分输出“烂片”。if-else 链按从高到低判断，边界值 9、7、5 分别归入较高的一档；对 0～10

之外的输入程序给出提示。

3. 判断回文数

实验内容：输入一个整数，将其各位数字反转后与原数比较。

代码：

```
// 练习 4-3 判断整数是否为回文数
// 将整数各位数字倒序重组，与原数比较。负数因带负号，不是回文数。
#include <iostream>
using namespace std;

int main()
{
    long long n;
    cout << "请输入一个整数：";
    cin >> n;

    if (n < 0) {
        cout << n << " 不是回文数。" << endl;
        return 0;
    }

    long long original = n;
    long long reversed = 0;
    while (n > 0) {
        reversed = reversed * 10 + n % 10;
        n /= 10;
    }

    if (original == reversed)
        cout << original << " 是回文数。" << endl;
    else
        cout << original << " 不是回文数。" << endl;

    return 0;
}
```

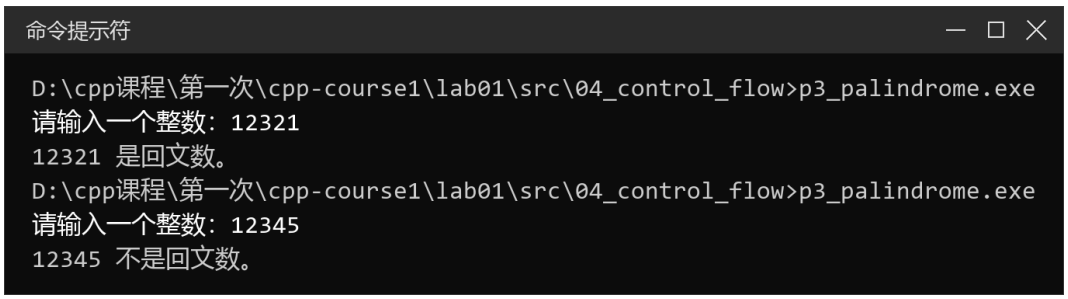


图 4-3 判断回文数运行结果

结果分析：12321 正读倒读相同，是回文数；12345 反转后为 54321，不是回文数。程序用 long long 保存反转结果以防止溢出，负数因带负号不按回文数处理。

4. 判断并输出水仙花数

实验内容：先判断输入的三位数是否为水仙花数，再输出全部三位水仙花数。

代码：

```
// 练习 4-4 水仙花数
// 先判断输入的三位数，再输出全部三位水仙花数。
#include <iostream>
using namespace std;

int main()
{
    int num;
    cout << "请输入一个三位整数：";
    cin >> num;

    if (num < 100 || num > 999) {
        cout << "输入的不是三位数。" << endl;
        return 1;
    }

    int a = num / 100;
    int b = num / 10 % 10;
    int c = num % 10;
    int sum = a * a * a + b * b * b + c * c * c;

    cout << num << (sum == num ? " 是" : " 不是") << "水仙花数。" << endl;

    cout << "所有三位水仙花数：";
    for (int i = 100; i <= 999; ++i) {
        int x = i / 100;
        int y = i / 10 % 10;
        int z = i % 10;
        if (x * x * x + y * y * y + z * z * z == i)
            cout << i << " ";
    }
    cout << endl;

    return 0;
}
```

```
命令提示符
D:\cpp课程\第一次\cpp-course1\lab01\src\04_control_flow>p4_narcissistic.exe
请输入一个三位整数：153
153 是水仙花数。
所有三位水仙花数：153 370 371 407
```

图 4-4 判断并输出水仙花数运行结果

结果分析：153 = 1^3 + 5^3 + 3^3，是水仙花数。遍历 100～999 后得到全部结果：153、370、371、407。

5. 构造 a，aa，aaa… 数列并求和

实验内容：输入项数 n 和数字 a，构造 n 项数列并求所有项之和。

代码：

```
// 练习 4-5 数列 a, aa, aaa, ... 前 n 项求和
// 第 i 项由前一项乘 10 再加 a 得到。
#include <iostream>
using namespace std;

int main()
{
    int n, a;
    cout << "请输入项数 n 和数字 a（1~9）：";
    cin >> n >> a;

    if (n <= 0 || a < 1 || a > 9) {
        cout << "输入不合法：n 应为正整数，a 应为 1~9。" << endl;
        return 1;
    }

    long long term = 0;
    long long sum = 0;
    for (int i = 1; i <= n; ++i) {
        term = term * 10 + a;
        sum += term;
        if (i > 1)
            cout << " + ";
        cout << term;
    }
    cout << " = " << sum << endl;

    return 0;
}
```



图 4-5 构造 a，aa，aaa… 数列并求和运行结果

结果分析：n = 3、a = 2 时数列为 2、22、222，和为 246。每一项由前一项乘 10 再加 a 递推得到，边构造边累加；和用 long long 保存以防溢出。

6. 输出 9×9 乘法口诀表

实验内容：用嵌套循环输出下三角形式的乘法口诀表。

代码：

```
// 练习 4-6 输出 9*9 乘法口诀表
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    for (int i = 1; i <= 9; ++i) {
        for (int j = 1; j <= i; ++j)
            cout << j << "*" << i << "=" << setw(2) << j * i << "  ";
        cout << endl;
    }
    return 0;
}
```

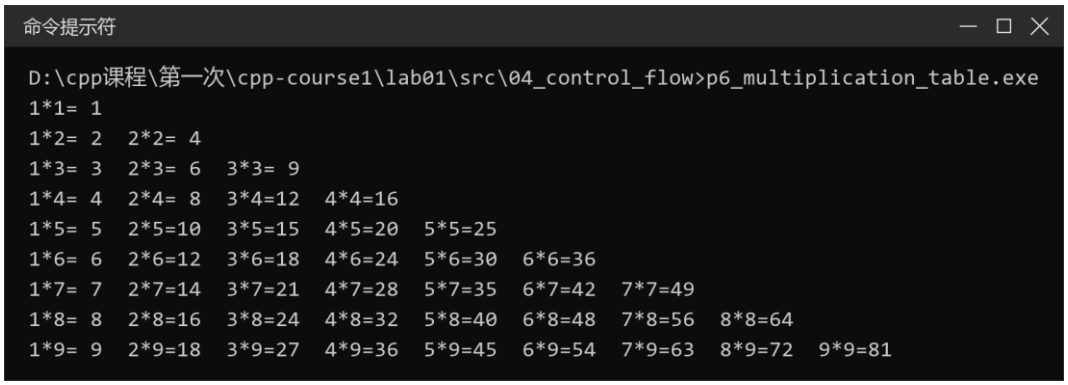


图 4-6 输出 9×9 乘法口诀表运行结果

结果分析：外层循环 i 控制行，内层循环 j 从 1 到 i，共输出 45 个等式，正好构成下三角。setw(2) 使乘积右对齐，表格整齐。

7. 兔子繁殖问题

实验内容：按“从第 3 个月起每月生一对”的规律计算 20 个月内每月的

兔子总对数。

代码：

```
// 练习 4-7 古典问题：兔子繁殖（斐波那契数列）
// 每对兔子从第 3 个月起每月生一对，求 20 个月内每月的兔子总对数。
#include <iostream>
using namespace std;

int main()
{
    long long a = 1, b = 1;   // 第 1、2 个月的兔子对数
    cout << "第 1 个月：1 对" << endl;
    cout << "第 2 个月：1 对" << endl;

    for (int month = 3; month <= 20; ++month) {
        long long c = a + b;
        cout << "第 " << month << " 个月：" << c << " 对" << endl;
        a = b;
        b = c;
    }

    cout << "20 个月时共有 " << b << " 对兔子。" << endl;
    return 0;
}
```

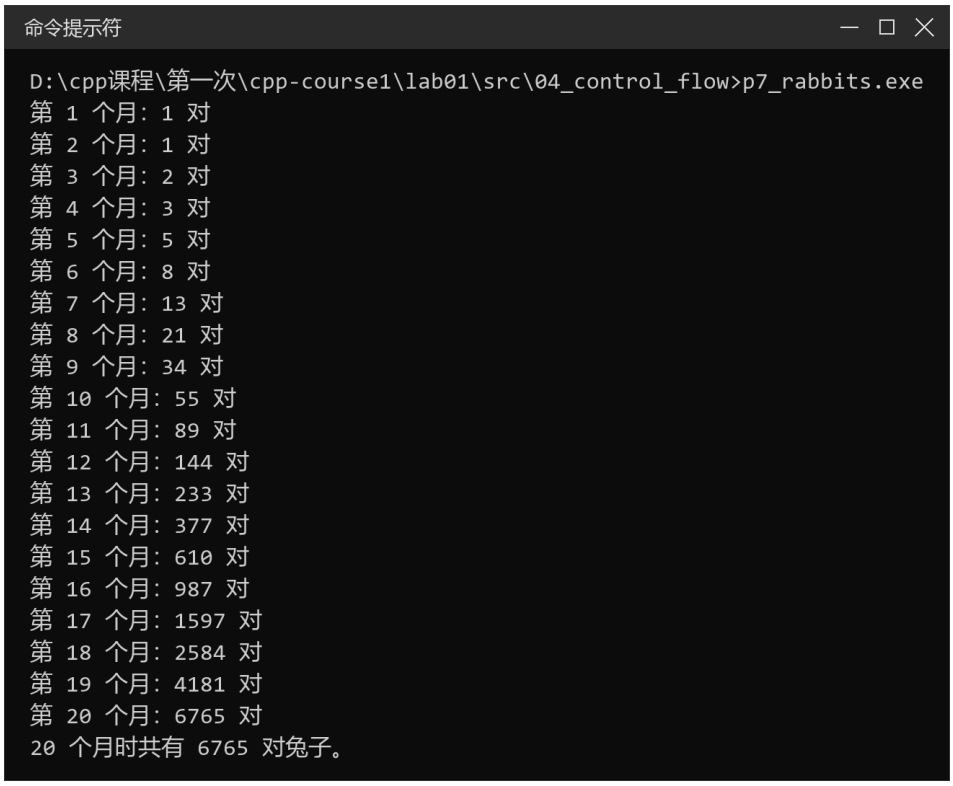


图 4-7 兔子繁殖问题运行结果

结果分析：第 1、2 个月各 1 对，以后每月等于前两个月之和，即斐波那契数列；第 20 个月共有 6765 对。循环中只保存前两项，空间复杂度为 O(1)。

8. 猴子吃桃问题

实验内容：已知第 10 天早上剩 1 个桃子，逆推第一天共摘了多少个。

代码：

```
// 练习 4-8 猴子吃桃
// 每天吃掉前一天剩下的一半再加一个，第 10 天早上剩 1 个。
// 逆推：前一天的数量 = (当天剩余 + 1) * 2。
#include <iostream>
using namespace std;

int main()
{
    int peaches = 1;   // 第 10 天早上剩下的桃子数

    for (int day = 9; day >= 1; --day) {
        peaches = (peaches + 1) * 2;
        cout << "第 " << day << " 天开始时有 " << peaches << " 个桃子" << endl;
    }

    cout << "第一天共摘了 " << peaches << " 个桃子。" << endl;
    return 0;
}
```



图 4-8 猴子吃桃问题运行结果

结果分析：逆推公式为 $x(d-1) = (x(d) + 1) \times 2$，逐日得到第 9 天 4 个、第 8 天 10 个……第 1 天 1534 个，所以第一天共摘了 1534 个桃子。

四、实验总结（本次实验的收获、体会、经验和教训）

本次实验完成了基本数据类型、算术与赋值运算符、关系/逻辑/位运算以及结构化控制结构四个部分的练习，共编写并运行 22 个程序。通过亲自动手编译、运行和观察输出，我对课堂上讲授的语法规则有了更具体的认识。

在数据类型方面，最深的体会是“取值范围”不是抽象概念。把 50000 赋给 short 后输出 −15536，让我真正理解了补码与溢出；用 sizeof 实测各类

型长度，也纠正了“int 一定是 4 字节、long 一定比 int 长”这类想当然的认识。常量用 const 修饰后不能被赋值，这一点通过取消注释、观察编译错误得到了验证。

在运算符方面，整数除法会截断小数、% 只能用于整数、余数的符号随被除数，这些细节都必须通过实际运行才能记牢。前置 ++ 与后置 ++ 的区别也在输出 5、7、7、5 的过程中变得清晰；而同一表达式中多次修改同一变量会被 g++ 警告为未定义行为，提醒我以后不能写依赖求值顺序的代码。位运算部分通过置位、清位、翻转的对照，理解了二进制层面的操作方式。

在控制结构方面，分段累进的奖金问题让我学会了“先判断区间、再累加各段”的写法；回文数、水仙花数、数列求和训练了循环与数位分解；兔子繁殖和猴子吃桃则分别对应正向递推和逆向递推两种思路。同一个问题换一个方向思考，往往能大幅简化程序。

本次实验也暴露了一些不足：编写循环时曾忽略边界条件，输入非法数据时程序缺少提示；部分输出没有对齐，可读性较差。后续实验中我会继续做到：变量定义时初始化、编译时打开 -Wall 检查警告、对输入做合法性判断，并在写代码前先想清楚算法步骤，再动手实现。